# Incident classifier benchmark: Laya (base vs fine-tuned) vs LNN

Qualcomm SM8750P dev kit (Snapdragon 8 Elite, Hexagon v79 NPU) · QAIRT/SNPE 2.47 · 11 Oct 2026

## 1. Summary

| Model | Best on-device build | Type accuracy | Relevance acc | Fields F1 | ms / turn | File |
|---|---|---|---|---|---|---|
| Base Laya (no fine-tuning, zero-shot) | W8A16 on NPU | 9.3% | — | — | 10.8 | 859 MB |
| **Fine-tuned Laya** | **W8A16 on NPU** | **87.6%** | **95.2%** | **0.661** | **10.9** | 860 MB |
| LNN (CfC, trained from scratch) | W8A8 on NPU | 57.5% | 74.7% | 0.047 | **0.96** | **9 MB** |

*Type accuracy over all 259 test turns; chance is 3.3%. File size includes the HTP graph prepared for SM8750.*

- **Fine-tuning is what makes Laya work:** 11.6% → 88.0% at full precision.
- **Fine-tuned Laya runs on the NPU with almost no loss:** 87.6% with 8-bit weights and 16-bit activations, against 88.0% unquantized on the same kit's CPU, and 47× faster (10.9 ms vs 510 ms).
- **Laya cannot use fully 8-bit activations;** that build collapses to chance. The LNN can (0.96 ms, no accuracy loss).
- **The LNN file is about 95× smaller and the model 11× faster, but 30 points less accurate,** and it barely detects form fields.
- **Department-first (two-stage) classification never beats flat classification** for any model.

**Recommendation:** fine-tuned Laya, W8A16 on the NPU, for accuracy. Its cost is size: about 380 MB of weights (860 MB file with the prepared graph). Next step: fine-tune the 6 MB Ettin-17m ( `bench/RESULTS.md` ) with the same notebook to see how much of Laya's accuracy a model about 60× smaller keeps.

## 2. Setup

**Data** ( `datagen/out` , see `datagen/doc/fine_tuning_dataset.pdf` ). One sample per user turn; input is the user's sentences so far (assistant text is never input). Every model uses the same split, by conversation:

| Split | Conversations | Turn samples | Used for |
|---|---|---|---|
| Train | 1,048 | 2,442 | fitting |
| Validation | 116 | 275 | picking the best epoch, confidence calibration |
| Test | 128 | 259 | every number in this document (held out; written with unseen reporters and places) |

**Outputs scored:** incident type (30 labels: 28 types + `undetermined` + `none` ), sentence relevance (relevant / filler) and form fields mentioned (60, multi-label). Base Laya has no heads, so only its type is scored.

**Two-stage (department first):** step 1 = department (Facilities, Housekeeping, Security, EHS, IT, Labs) or `undetermined` / `none` ; step 2 = the best type inside that department. For base Laya, from embedding similarity to department descriptions. For the classifiers, from the type head (department probability = sum of its types' probabilities). Same single forward pass.

**Builds compared** (all on the dev kit except where noted):

| Build | Where | Precision |
|---|---|---|
| fp32 | Kryo CPU (base Laya also Adreno GPU) | unquantized |
| W8A16 | Hexagon v79 NPU | 8-bit weights, 16-bit activations, per-channel, 32-bit bias |
| W8A8 | Hexagon v79 NPU | 8-bit weights and activations, per-channel, 32-bit bias |

All NPU builds: calibrated on 50 training samples, HTP graph prepared offline for SM8750, no CPU fallback, burst performance mode.

## 3. Models

| | Base Laya | Fine-tuned Laya | LNN |
|---|---|---|---|
| Architecture | ModernBERT-large encoder, 28 layers, d=1024 | same + LoRA (r=16) merged + 3 heads | bidirectional CfC, hidden 192, embedding 256 |
| Parameters | 395M | 395M (7.3M trained) | 1.56M (all trained) |
| Vocabulary | 50,368 | 50,368 | 4,118: training-split tokens only; 8.7% of test tokens are unknown |
| Training | none (published weights, revision `7b928d8`) | 6 epochs on a Colab T4, LoRA lr 3e-4, heads 1e-3 | 39 epochs on the laptop CPU (445 s), early stopping (best = epoch 33) |
| Calibration temperature (validation) | — | 1.57 | fitted, stored in `lnn.pt` |

## 4. Full results (259 test turns)

| Model | Build | Type acc (all turns) | Final turn | Macro-F1 | Two-stage | Stage-1 dept | Relevance | Filler F1 | Fields F1 | ms / turn | Init ms | File MB |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Base Laya | fp32 CPU | 11.6% | 11.7% | 0.077 | 8.9% | 10.0% | — | — | — | 495 | 506 | 1,507 |
| Base Laya | fp32 GPU | 12.4% | 11.7% | 0.086 | 8.9% | 10.0% | — | — | — | 80 | 3,266 | 1,507 |
| Base Laya | W8A16 NPU | 9.3% | 7.8% | 0.057 | 8.5% | 8.5% | — | — | — | 10.8 | 107 | 859 |
| Base Laya | W8A8 NPU | 4.2% | 5.5% | 0.028 | 4.2% | 13.1% | — | — | — | 6.8 | 87 | 759 |
| **Fine-tuned Laya** | fp32 CPU | **88.0%** | 89.8% | 0.886 | 87.6% | 89.6% | 95.5% | 0.859 | 0.659 | 510 | 476 | 1,507 |
| **Fine-tuned Laya** | **W8A16 NPU** | **87.6%** | **90.6%** | **0.882** | 87.3% | 89.2% | **95.2%** | **0.850** | **0.661** | **10.9** | 107 | 860 |

| Model | Build | Type acc (all turns) | Final turn | Macro-F1 | Two-stage | Stage-1 dept | Relevance | Filler F1 | Fields F1 | ms / turn | Init ms | File MB |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Fine-tuned Laya | W8A8 NPU | 1.9% | 1.6% | 0.005 | 3.1% | 26.6% | 59.1% | 0.179 | 0.014 | 6.8 | 91 | 759 |
| LNN | fp32 CPU | 57.1% | 53.1% | 0.515 | 56.8% | 69.5% | 74.7% | 0.486 | 0.047 | 355 | 1,331 | 9 |
| LNN | W8A16 NPU | 57.5% | 53.9% | 0.517 | 57.1% | 69.9% | 74.5% | 0.484 | 0.047 | 1.6 | 29 | 12 |
| **LNN** | **W8A8 NPU** | **57.5%** | 53.9% | 0.518 | 57.1% | 69.9% | 74.7% | 0.486 | 0.047 | **0.96** | 25 | **9** |

Cross-checks: fine-tuned Laya fp32 on the kit equals the Colab GPU result exactly (88.0%, relevance 95.5%, fields 0.659); base Laya fp32 on the kit equals the laptop; the LNN on the kit equals the laptop (57.1%).

## 5. Findings

**Fine-tuning.** Base Laya's embeddings do not encode the incident type: they are nearly identical for every input (mean cosine 0.90), and even step 1 of the two-stage method (8 options) is at chance. LoRA fine-tuning of 7.3M parameters on 2,442 samples took it to 88%, with sentence relevance at 95% and form-field detection F1 0.66.

**Quantization.**

1. *Attention mask (fixed for every build).* ModernBERT adds `float32.min` (−3.4e38) for masked positions. In a quantized graph that one value destroys the range of the attention scores and every input gets nearly the same output. Exporting with −60 instead (identical in fp32, max difference 0.0) fixed it: base W8A16 went from cosine 0.14 to 0.985 with the fp32 embedding, and from 18.8 to 10.8 ms.
2. *8-bit activations.* Laya (base and fine-tuned) collapses to chance with 8-bit activations, even with the mask fix, per-channel weights and 32-bit biases: its activations have outliers that one 8-bit range cannot hold. **W8A16 is lossless** (fine-tuned: 87.6% vs 88.0%). The small LNN quantizes cleanly even at W8A8.
3. *Multiple outputs (fixed).* SNPE on the NPU only wrote the last of the three output tensors, so the classifiers are exported with one concatenated output, split again by the scorer.

**Two-stage (department first).** It never helps: fine-tuned Laya 87.3% vs 87.6% flat, the LNN 57.1% vs 57.5%, base Laya 8.9% vs 11.6%. A flat 30-way head already ranks departments implicitly; forcing a hard department choice first adds a second place to make a mistake.

**LNN.** Trained from scratch on 2,442 samples, it overfits (training loss 0.016 against 45% validation accuracy) and has no knowledge of English beyond the training split (8.7% of test tokens unknown). It learns the incident type partly (57.5%) and relevance weakly (74.7%), but almost never finds form fields (F1 0.047). Its strengths are size (9 MB) and speed (0.96 ms, fully 8-bit).

**Speed and size.** The NPU makes the big model practical: 10.9 ms per turn versus 510 ms on the kit's CPU and 80 ms on its GPU. Laya's cost is memory: about 380 MB of 8-bit weights (860 MB file with the prepared graph), plus about 0.1 s first load.

## 6. Test-time shuffling and strategies for >95% per conversation

Model: fine-tuned Laya, W8A16 on the NPU (the shipped build), same 259 test turns / 128 conversations. "Final turn" = the decision at the end of each conversation, which is what the report form uses.

### 6a. Shuffle-and-vote (test-time augmentation)

Each input (the user's turns so far) was shuffled 5 times: the order of turns and of the sentences inside each turn, turn separators kept (seeded, `laya/tta.py`). 182 of 259 turns have more than one sentence, so their shuffles differ. 1,295 NPU runs took 15 s.

| Method | All turns | Final turn |
|---|---|---|
| **Single input, original order** | **87.6%** | **90.6%** |
| 5 shuffles, majority vote (tie → highest summed probability) | 84.9% | 86.7% |
| 5 shuffles, mean probability | 85.3% | 86.7% |
| Original + 4 shuffles, majority vote | 86.5% | 88.3% |
| One shuffled view alone | 84.6% | 85.2% |

**Shuffling hurts.** The model was trained on conversations in their real order, where the latest turn usually carries the decisive detail ("it's actually the fourth floor", "turns out it's the ceiling pipe"). Shuffled inputs are out of distribution (a single shuffled view drops to 84.6%), and voting over five weaker views cannot beat one stronger one: the views make correlated mistakes. Test-time augmentation only helps when the augmentation is a transformation the model was trained to be invariant to.

### 6b. What does work: measured on the same outputs

| Strategy | Conversations answered directly | Final-decision accuracy |
|---|---|---|
| Always answer (baseline) | 100% | 90.6% |
| Right type is in the **top-2** | — | 96.1% |
| Right type is in the **top-3** | — | 97.7% |
| Answer if confidence ≥ 0.8, otherwise confirm between top-2 | 73.4% (96.8% correct) | 94.5% |
| **Answer if confidence ≥ 0.8, otherwise offer the top-3 to pick from** | 73.4% (96.8% correct) | **96.1%** |
| "Sticky": lock in the first type predicted with ≥ 0.9 confidence | 100% | 75.8% (worse) |

*Confidence = calibrated probability (temperature 1.57 from validation). 128 conversations: ±3–4 points is noise. The 0.8 threshold was read off these test results, which flatters it; for the product, choose it on the validation split and confirm on new data.*

**Where the 32 wrong turns come from:** 11 are "undetermined vs a type", i.e. *when* to commit, not *which* type. Accuracy when neither label is undetermined is 90.7%. The most common type mix-ups are lab equipment → printer/device, lighting → conference-room AV, theft → suspicious person.

### 6c. Strategies to reach >95% within a conversation (ranked)

1. **Confidence gate + 3-option confirmation (measured: 96.1%).** Answer directly when confident (≥ 0.8); otherwise the assistant asks "Is this about A, B or C?". About 1 in 4 conversations gets one extra tap. This extends the taxonomy's existing rule (confidence < 0.6 → confirm top-2) and needs no retraining.
2. **Keep asking before committing.** Many low-confidence turns are early "undetermined" ones; the app is already asking follow-up questions, so the gate should apply to the final decision, not to every turn. The answer is committed only when the form is submitted.
3. **Targeted data for the confusable pairs above** (lab equipment vs device, lighting vs AV, theft vs suspicious), plus a consistency pass on when `undetermined` switches to a type (a third of the errors).

4. **Real ensembles instead of input shuffles:** 2–3 fine-tunes with different seeds, averaging probabilities. Weight diversity gives independent errors; shuffled inputs do not. Costs 2–3× NPU time (~22–33 ms) and memory, so it suits a smaller base model (Ettin) better than Laya.
5. **Train with order augmentation, then shuffle at test time.** If shuffled orders are part of training, TTA stops hurting and can help, at 5× inference cost.
6. **Use the field head as evidence.** Fields that only one type uses (`persons_trapped`, `pest_kind`, `av_equipment`...) are strong type signals; combining them with the type head, tuned on validation, can fix type confusions.
7. **Stronger fine-tune:** full fine-tuning instead of LoRA, more epochs with early stopping, label smoothing. Each needs a validation run; gains are typically a few points.
8. **Real user data:** the biggest unknown is not the model but the test set. A team-written, spoken test set will show the real number to optimise against.

## 7. Limits

- **259 test turns from 128 conversations:** differences under about 3–4 points are within noise.
- **Synthetic data:** the test set is LLM-written like the training data, so absolute accuracy is optimistic. A team-written test set, ideally spoken and transcribed, is needed before trusting these numbers for the product.
- **One training run each,** no hyperparameter search. The LNN in particular could improve with regularisation, a larger vocabulary or pretrained embeddings; it would not close a 30-point gap without pretraining.
- **Timing is model execution on the device only;** tokenization and app overhead are not included.
- **Fully unquantized Laya cannot run on the NPU in this SDK path** (the SNPE HTP runtime is fixed-point), so "unquantized on the device" means the CPU (and GPU for base Laya).

## 8. Reproduce

```
source qairt-env.sh
# base Laya
python laya/export.py base && qairt-converter --input_network laya/out/base/model.onnx --
output_path laya/out/base/model.dlc
# fine-tuned Laya (laya_ft.pt from laya/laya_finetune.ipynb on Colab)
python laya/export.py ft laya_ft.pt && qairt-converter --input_network laya/out/ft/model.onnx --
output_path laya/out/ft/model.dlc
# LNN: trains on CPU and exports
python laya/train_lnn.py && qairt-converter --input_network laya/out/lnn/model.onnx --output_path
laya/out/lnn/model.dlc
# for each of base / ft / lnn:
laya/quant_variant.sh <mode> w8a16 --act_bitwidth 16 --bias_bitwidth 32 --
use_per_channel_quantization
laya/quant_variant.sh <mode> w8a8  --use_per_channel_quantization --bias_bitwidth 32
ANDROID_SERIAL=fe7980db laya/run_device.sh <mode> w8a16   # also: w8a8, fp32cpu, fp32gpu
python laya/score.py <mode> w8a16
# shuffle-and-vote + strategy analysis (fine-tuned model)
python laya/tta.py build && ANDROID_SERIAL=fe7980db laya/run_device.sh ft w8a16 tta && python
laya/tta.py score w8a16
```